Lecture 26

Parameters, optimization, backprop

COURSE MATH 637: Mathematical Techniques in Data Science

WEBSITE https://jacobcalvert.com/teaching/math637fall2026/

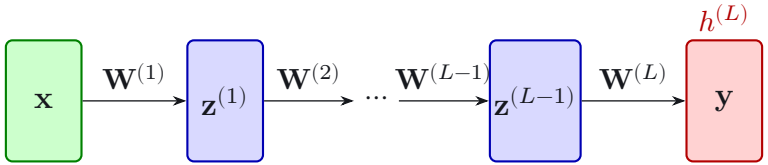
INSTRUCTOR Jacob Calvert (calvert@udel.edu)

OFFICE HOURS MW 2:45pm–3:45pm, Ewing Hall 509

Plan for today

• Recap	2 min
• Large networks and their parameters	6 min
• Error functions	4 min
• Optimization	12 min
• Backpropagation	16 min
• PyTorch	8 min

Recall: a neural net is function composition



$$\mathbf{z}^{(l)} = h^{(l)}\big(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)}\big), \qquad \mathbf{z}^{(0)} = \mathbf{x}, \quad \mathbf{y} = \mathbf{z}^{(L)}$$

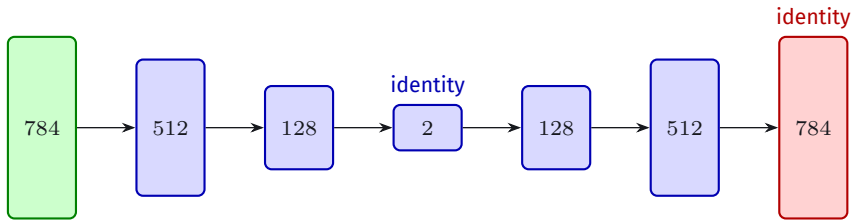
- hidden layers use ReLU (not written above the block)
- the output activation $h^{(L)}$ matches the target: identity, σ, or softmax
- each $\mathbf{W}^{(l)}$ holds one layer's weights; column 0 holds the biases

Plan for today

• Recap	2 min
• Large networks and their parameters	6 min
• Error functions	4 min
• Optimization	12 min
• Backpropagation	16 min
• PyTorch	8 min

Example: a larger network

A 28×28 MNIST image $=$ vector w/ $D = 784$ entries. A network for encoding such images:



- blocks $=$ layers labeled by width, arrows $=$ weight matrices $\mathbf{W}^{(l)}$
- activation functions are ReLU unless labeled otherwise

Counting parameters: a unit, a layer

One unit with q inputs

- weights $w_{j1}, \ldots, w_{jq}$ and bias w_{j0}: $q + 1$
- the activation has no parameters

One layer: q inputs, m units

- m units, each with $q + 1$: $m(q + 1)$
- the size of $\mathbf{W}^{(l)} \in \mathbb{R}^{m \times (q+1)}$: the $+1$ is the column for the constant 1

Counting parameters: a unit, a layer

One unit with q inputs

- weights $w_{j1}, \ldots, w_{jq}$ and bias w_{j0}: $q+1$
- the activation has no parameters

One layer: q inputs, m units

- m units, each with $q+1$: $m(q+1)$
- the size of $\mathbf{W}^{(l)} \in \mathbb{R}^{m \times (q+1)}$: the $+1$ is the column for the constant 1

A network with widths $d_0, d_1, \ldots, d_L$

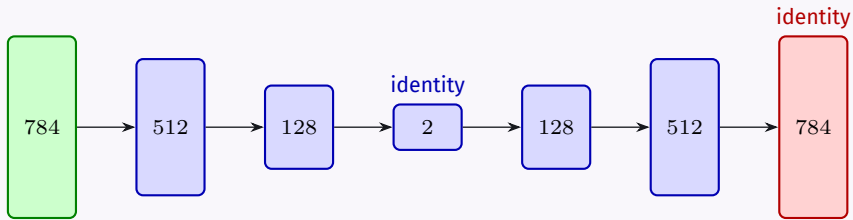
$$P = \sum_{l=1}^{L} d_l \left(d_{l-1} + 1\right)$$

- $P =$ the number of parameters

Exercise: counting parameters

Question

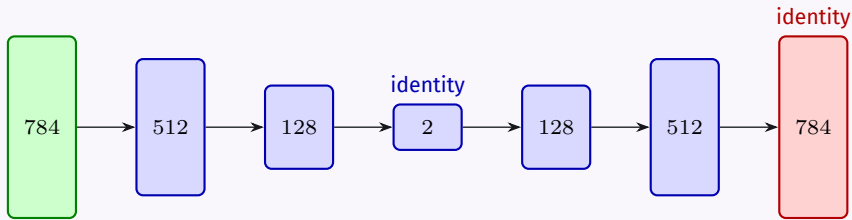
How many parameters does this network have?



Exercise: counting parameters

Question

How many parameters does this network have?



Answer

$$P = 512 \cdot 785 + 128 \cdot 513 + 2 \cdot 129 + 128 \cdot 3 + 512 \cdot 129 + 784 \cdot 513 = 936{,}466$$

Plan for today

• Recap	2 min
• Large networks and their parameters	6 min
• Error functions	4 min
• Optimization	12 min
• Backpropagation	16 min
• PyTorch	8 min

Error functions

Note: Given example $\mathbf{x}_n$ with label of "target" t_n or $\mathbf{t}_n$, output of network with weights $\mathbf{w}$ is denoted y_n or $\mathbf{y}_n$.

task	output activation	$E_n(\mathbf{w})$
regression	identity	$\frac{1}{2}(y_n - t_n)^2$
two classes, $t_n \in \{0, 1\}$	σ	$-t_n \ln y_n - (1 - t_n) \ln(1 - y_n)$
K classes, $\mathbf{t}_n$ one-hot	softmax	$-\sum_k t_{nk} \ln y_{nk}$
dimension reduction	identity	$\frac{1}{2} \left\| \mathbf{y}_n - \mathbf{x}_n \right\|^2$

Error functions

Note: Given example $\mathbf{x}_n$ with label of "target" t_n or $\mathbf{t}_n$, output of network with weights $\mathbf{w}$ is denoted y_n or $\mathbf{y}_n$.

task	output activation	$E_n(\mathbf{w})$
regression	identity	$\frac{1}{2}(y_n - t_n)^2$
two classes, $t_n \in \{0, 1\}$	σ	$-t_n \ln y_n - (1 - t_n) \ln(1 - y_n)$
K classes, $\mathbf{t}_n$ one-hot	softmax	$-\sum_k t_{nk} \ln y_{nk}$
dimension reduction	identity	$\frac{1}{2} \left\| \mathbf{y}_n - \mathbf{x}_n \right\|^2$

Key idea: The task tells you what output activation and error to use.

The training problem

Stack all weights into one vector $\mathbf{w}$, with P entries.

Problem (Training)

$$\text{minimize} \quad E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}) \quad \text{over } \mathbf{w} \in \mathbb{R}^P$$

- Example: $P = 936{,}466$ for the encoder–decoder network

The training problem

Stack all weights into one vector $\mathbf{w}$, with P entries.

Problem (Training)

$$\text{minimize} \quad E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}) \quad \text{over } \mathbf{w} \in \mathbb{R}^P$$

- Example: $P = 936{,}466$ for the encoder–decoder network
- Issue 1: unlike linear regression, no closed form

The training problem

Stack all weights into one vector $\mathbf{w}$, with P entries.

Problem (Training)

$$\text{minimize} \quad E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}) \quad \text{over } \mathbf{w} \in \mathbb{R}^P$$

- Example: $P = 936{,}466$ for the encoder–decoder network
- Issue 1: unlike linear regression, no closed form
- Issue 2: Newton's method needs a $P \times P$ Hessian, which has $936{,}466^2 \approx 9 \times 10^{11}$ entries in our example

The training problem

Stack all weights into one vector $\mathbf{w}$, with P entries.

Problem (Training)

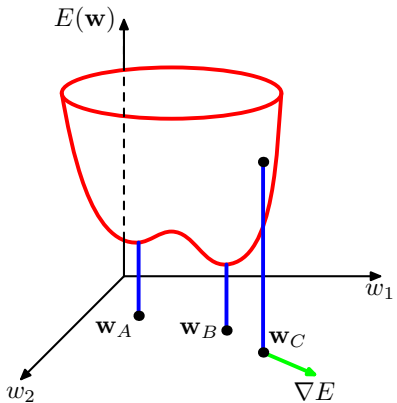
$$\text{minimize} \quad E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}) \quad \text{over } \mathbf{w} \in \mathbb{R}^P$$

- Example: $P = 936{,}466$ for the encoder–decoder network
- Issue 1: unlike linear regression, no closed form
- Issue 2: Newton's method needs a $P \times P$ Hessian, which has $936{,}466^2 \approx 9 \times 10^{11}$ entries in our example
- Issue 3: E likely not convex (consider exchanging two hidden units, with their weights)

Plan for today

• Recap	2 min
• Large networks and their parameters	6 min
• Error functions	4 min
• Optimization	12 min
• Backpropagation	16 min
• PyTorch	8 min

The error surface



Bishop and Bishop, Fig. 7.1

- $E(\mathbf{w})$ is a surface over weight space $\mathbb{R}^P$
- $E(\mathbf{w} + \delta\mathbf{w}) \approx E(\mathbf{w}) + \delta\mathbf{w}^\mathsf{T}\nabla E(\mathbf{w})$, so $-\nabla E$ is direction of steepest descent
- $\nabla E(\mathbf{w}) = \mathbf{0}$ can give minima, maxima, saddle points
- $\mathbf{w}_A =$ a local minimum; $\mathbf{w}_B =$ global min

Batch gradient descent

At step τ, with learning rate $\eta > 0$, do:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)}), \qquad \nabla E = \sum_{n=1}^{N} \nabla E_n.$$

Key idea: Every step of batch gradient descent uses all N training examples, requiring N gradient evaluations. When N is large this is slow.

- Example: MNIST has $N = 10{,}000$, so $10{,}000$ gradients must be calculated for one step of batch gradient descent.

The other extreme: stochastic gradient descent

Algorithm 7.1: Stochastic gradient descent

Input: Training set of data points indexed by $n \in \{1, \ldots, N\}$
Error function per data point $E_n(\mathbf{w})$
Learning rate parameter η
Initial weight vector $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$
repeat
  $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_n(\mathbf{w})$ `// update weight vector`
  $n \leftarrow n + 1 (\text{mod } N)$ `// iterate over data`
until convergence
return $\mathbf{w}$

Bishop and Bishop, Algorithm 7.1

Key idea: Use only example per step, hence one gradient evaluation per step. This gives "noisy" steps, since $\nabla E_n \approx \nabla E$.

Happy medium: Mini-batch SGD

Algorithm 7.2: Mini-batch stochastic gradient descent

Input: Training set of data points indexed by $n \in \{1, \ldots, N\}$
Batch size B
Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
Learning rate parameter η
Initial weight vector $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$
repeat
  $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n+B-1}(\mathbf{w})$ `// weight vector update`
  $n \leftarrow n + B$
  if $n > N$ **then**
    shuffle data
    $n \leftarrow 1$
  end if
until convergence
return $\mathbf{w}$

Bishop and Bishop, Algorithm 7.2

Key idea: Use B examples per step. Note that $B = 1$ is SGD, $B = N$ is batch gradient descent.

- the gradient's standard deviation falls like $1/\sqrt{B}$, but cost grows like B
- MNIST with $B = 64$ requires 157 updates per epoch

Now add momentum

Algorithm 7.3: Stochastic gradient descent with momentum

Input: Training set of data points indexed by $n \in \{1, \ldots, N\}$
Batch size B
Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
Learning rate parameter η
Momentum parameter μ
Initial weight vector $\mathbf{w}$
Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$
$\Delta\mathbf{w} \leftarrow \mathbf{0}$
repeat
  $\Delta\mathbf{w} \leftarrow -\eta\nabla E_{n:n+B-1}(\mathbf{w}) + \mu\Delta\mathbf{w}$ // calculate update term
  $\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w}$ // weight vector update
  $n \leftarrow n + B$
  if $n > N$ **then**
    shuffle data
    $n \leftarrow 1$
  end if
until convergence
return $\mathbf{w}$

Bishop and Bishop, Algorithm 7.3

Key idea: $\Delta\mathbf{w}$ accumulates past steps. A constant gradient gives steps tending to $-\eta\nabla E/(1-\mu)$, ten times larger at $\mu = 0.9$, while oscillating steps cancel.

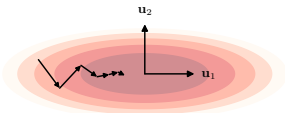


Fig. 7.6

Adaptive moment estimation (Adam)

Algorithm 7.4: Adam optimization

Input: Training set of data points indexed by $n \in \{1, \ldots, N\}$
Batch size B
Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
Learning rate parameter η
Decay parameters β_1 and β_2
Stabilization parameter δ
Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$
$\mathbf{s} \leftarrow \mathbf{0}$
$\mathbf{r} \leftarrow \mathbf{0}$
repeat
  Choose a mini-batch at random from $\mathcal{D}$
  $\mathbf{g} = -\nabla E_{n:n+B-1}(\mathbf{w})$ // evaluate gradient vector
  $\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1 - \beta_1)\mathbf{g}$
  $\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1 - \beta_2)\mathbf{g} \odot \mathbf{g}$ // element-wise multiply
  $\widehat{\mathbf{s}} \leftarrow \mathbf{s}/(1 - \beta_1^\tau)$ // bias correction
  $\widehat{\mathbf{r}} \leftarrow \mathbf{r}/(1 - \beta_2^\tau)$ // bias correction
  $\Delta \mathbf{w} \leftarrow -\eta \frac{\widehat{\mathbf{s}}}{\sqrt{\widehat{\mathbf{r}}} + \delta}$ // element-wise operations
  $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$ // weight vector update
  $n \leftarrow n + B$
  if $n + B > N$ **then**
    shuffle data
    $n \leftarrow 1$
  end if
until convergence
return $\mathbf{w}$

Bishop and Bishop, Algorithm 7.4

Key idea: A step size for each weight based on s, which averages the gradient, and its square r its square. Each step size is about η in every coordinate, whatever the scale of the gradient.

Adaptive moment estimation (Adam)

Algorithm 7.4: Adam optimization

Input: Training set of data points indexed by $n \in \{1, \ldots, N\}$
Batch size B
Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$
Learning rate parameter η
Decay parameters β_1 and β_2
Stabilization parameter δ
Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$
$\mathbf{s} \leftarrow \mathbf{0}$
$\mathbf{r} \leftarrow \mathbf{0}$
repeat
  Choose a mini-batch at random from $\mathcal{D}$
  $\mathbf{g} = -\nabla E_{n:n+B-1}(\mathbf{w})$ // evaluate gradient vector
  $\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1 - \beta_1)\mathbf{g}$
  $\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1 - \beta_2)\mathbf{g} \odot \mathbf{g}$ // element-wise multiply
  $\widehat{\mathbf{s}} \leftarrow \mathbf{s}/(1 - \beta_1^{\uparrow})$ // bias correction
  $\widehat{\mathbf{r}} \leftarrow \mathbf{r}/(1 - \beta_2^{\uparrow})$ // bias correction
  $\Delta\mathbf{w} \leftarrow -\eta \dfrac{\widehat{\mathbf{s}}}{\sqrt{\widehat{\mathbf{r}}} + \delta}$ // element-wise operations
  $\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w}$ // weight vector update
  $n \leftarrow n + B$
  if $n + B > N$ **then**
    shuffle data
    $n \leftarrow 1$
  end if
until convergence
return $\mathbf{w}$

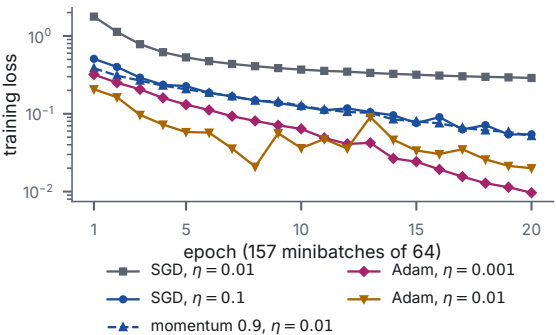
Bishop and Bishop, Algorithm 7.4

Key idea: A step size for each weight based on s, which averages the gradient, and its square r its square. Each step size is about η in every coordinate, whatever the scale of the gradient.

Watch out: Some typos in Bishop–Bishop: use $\mathbf{g} = \nabla E$ instead and if-statement should use $n > N$ instead of $n + B > N$.

Example: comparison of five optimizers

MNIST, 784–128–10, $B = 64$, 20 epochs, one run each:



optimizer	test accuracy
SGD, $\eta = 0.01$	92.6%
SGD, $\eta = 0.1$	96.55%
momentum, $\eta = 0.01$	96.35%
Adam, $\eta = 0.001$	96.6%
Adam, $\eta = 0.01$	95.45%

- SGD at 0.01 is far from fitting
- momentum at 0.01 tracks SGD at $0.1 = 0.01/(1 - 0.9)$

How to calculate the gradients?

Every update needs the P derivatives $\partial_{w_i} E_n(\mathbf{w})$ on a mini-batch.

Finite differences

- for each i calculate $(E_n(\mathbf{w} + \epsilon \mathbf{e}_i) - E_n(\mathbf{w}))/\epsilon$ for small ϵ
- requires $P + 1$ evaluations of the network

Backpropagation

- all P derivatives, by the chain rule
- cost is a small multiple of one evaluation

How to calculate the gradients?

Every update needs the P derivatives $\partial_{w_i} E_n(\mathbf{w})$ on a mini-batch.

Finite differences

- for each i calculate $(E_n(\mathbf{w} + \epsilon \mathbf{e}_i) - E_n(\mathbf{w}))/\epsilon$ for small ϵ
- requires $P + 1$ evaluations of the network

Backpropagation

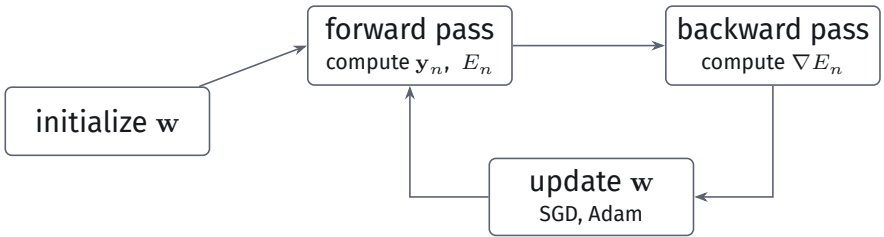
- all P derivatives, by the chain rule
- cost is a small multiple of one evaluation

- SGD, mini-batches, momentum, and Adam need only the gradient, however it is computed

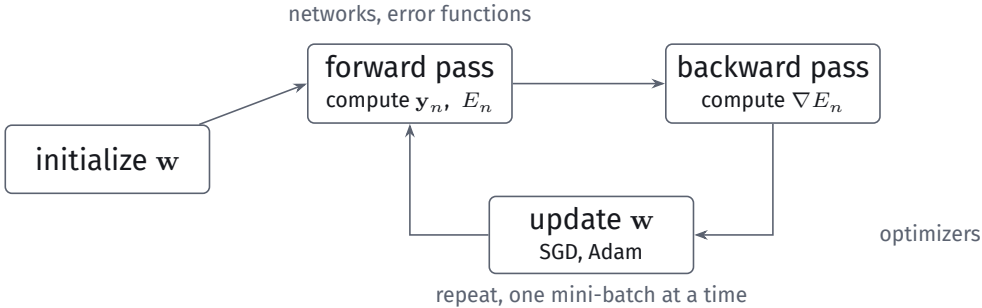
Plan for today

• Recap	2 min
• Large networks and their parameters	6 min
• Error functions	4 min
• Optimization	12 min
• Backpropagation	16 min
• PyTorch	8 min

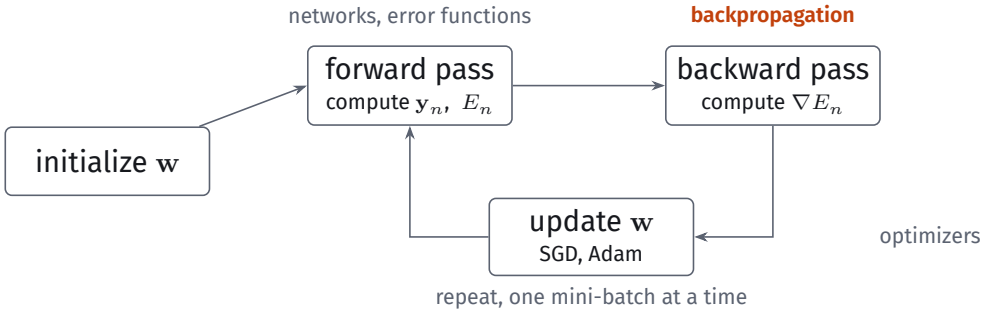
The training loop



The training loop

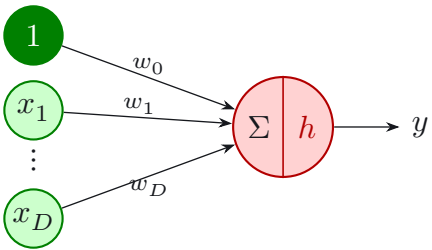


The training loop



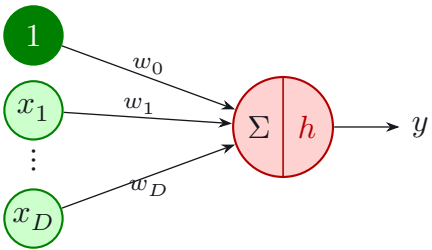
- backpropagation: all P derivatives, in one pass backward through the network

One neuron: error times input



- E_n depends on w_i only through $a = w_0 x_0 + \cdots + w_D x_D$

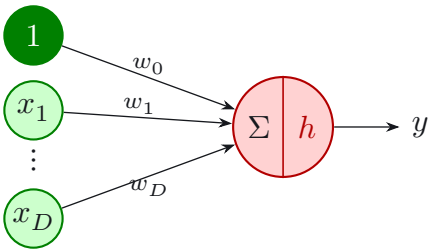
One neuron: error times input



- E_n depends on w_i only through $a = w_0 x_0 + \cdots + w_D x_D$
- chain rule:

$$\frac{\partial E_n}{\partial w_i} = \frac{\partial E_n}{\partial a} \frac{\partial a}{\partial w_i} = \delta \, x_i$$

One neuron: error times input

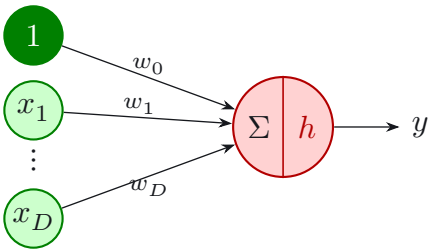


- E_n depends on w_i only through $a = w_0 x_0 + \cdots + w_D x_D$
- chain rule:

$$\frac{\partial E_n}{\partial w_i} = \frac{\partial E_n}{\partial a} \frac{\partial a}{\partial w_i} = \delta \, x_i$$

- $\delta = \partial E_n / \partial a$ is the error of the unit

One neuron: error times input

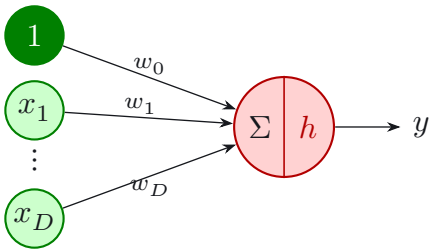


- E_n depends on w_i only through $a = w_0 x_0 + \cdots + w_D x_D$
- chain rule:

$$\frac{\partial E_n}{\partial w_i} = \frac{\partial E_n}{\partial a} \frac{\partial a}{\partial w_i} = \delta \, x_i$$

- $\delta = \partial E_n / \partial a$ is the error of the unit
- Example: $h(a) = a$, $E_n = \frac{1}{2}(y - t)^2$ give $\delta = y - t$

One neuron: error times input



- E_n depends on w_i only through $a = w_0 x_0 + \cdots + w_D x_D$
- chain rule:

$$\frac{\partial E_n}{\partial w_i} = \frac{\partial E_n}{\partial a} \frac{\partial a}{\partial w_i} = \delta \, x_i$$

- $\delta = \partial E_n / \partial a$ is the error of the unit
- Example: $h(a) = a$, $E_n = \frac{1}{2}(y - t)^2$ give $\delta = y - t$

Key idea: Derivative for a weight = error at output end $\times$ value at input end.

The error of an output unit

task	output act.	$E_n(\mathbf{w})$
regression	identity	$\frac{1}{2}(y_n - t_n)^2$
two classes	σ	$-t_n \ln y_n - (1 - t_n) \ln(1 - y_n)$
K classes	softmax	$-\sum_k t_{nk} \ln y_{nk}$
dimension reduction	identity	$\frac{1}{2} \left\| \mathbf{y}_n - \mathbf{x}_n \right\|^2$

The error of an output unit

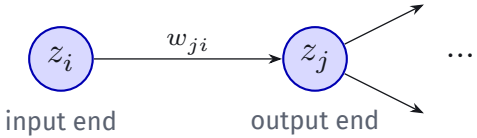
task	output act.	$E_n(\mathbf{w})$	$\delta_k = \partial E_n / \partial a_k$
regression	identity	$\frac{1}{2}(y_n - t_n)^2$	$y_n - t_n$
two classes	σ	$-t_n \ln y_n - (1 - t_n) \ln(1 - y_n)$	$y_n - t_n$
K classes	softmax	$-\sum_k t_{nk} \ln y_{nk}$	$y_{nk} - t_{nk}$
dimension reduction	identity	$\frac{1}{2} \left\| \mathbf{y}_n - \mathbf{x}_n \right\|^2$	$y_{nk} - x_{nk}$

The error of an output unit

task	output act.	$E_n(\mathbf{w})$	$\delta_k = \partial E_n / \partial a_k$
regression	identity	$\frac{1}{2}(y_n - t_n)^2$	$y_n - t_n$
two classes	σ	$-t_n \ln y_n - (1 - t_n) \ln(1 - y_n)$	$y_n - t_n$
K classes	softmax	$-\sum_k t_{nk} \ln y_{nk}$	$y_{nk} - t_{nk}$
dimension reduction	identity	$\frac{1}{2} \left\| \mathbf{y}_n - \mathbf{x}_n \right\|^2$	$y_{nk} - x_{nk}$

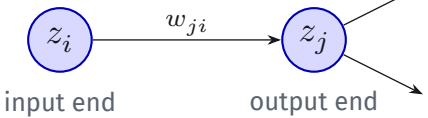
Key idea: For each task, the output error is "prediction minus target."

Any weight in any network



- unit j: $a_j = \sum_i w_{ji} z_i$, $z_j = h(a_j)$

Any weight in any network

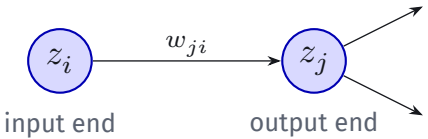


…

- unit j: $a_j = \sum_i w_{ji} z_i$, $z_j = h(a_j)$
- E_n depends on w_{ji} only through a_j, so

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j \, z_i, \qquad \delta_j = \frac{\partial E_n}{\partial a_j}$$

Any weight in any network



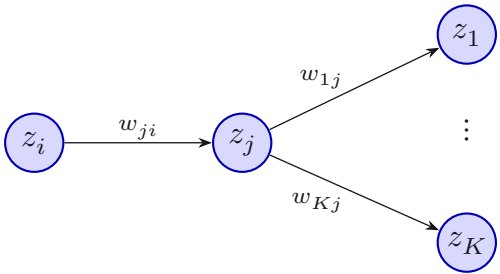
…

- unit j: $a_j = \sum_i w_{ji} z_i$, $z_j = h(a_j)$
- E_n depends on w_{ji} only through a_j, so

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j \, z_i, \qquad \delta_j = \frac{\partial E_n}{\partial a_j}$$

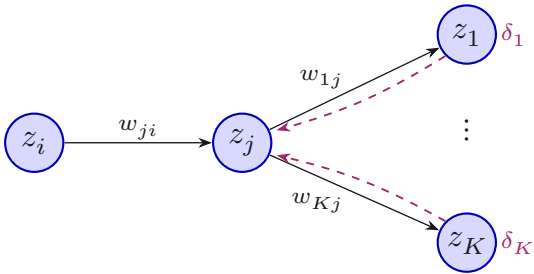
- the same rule as for one neuron
- z_i is known from the forward pass; only δ_j is missing

The error of a hidden unit



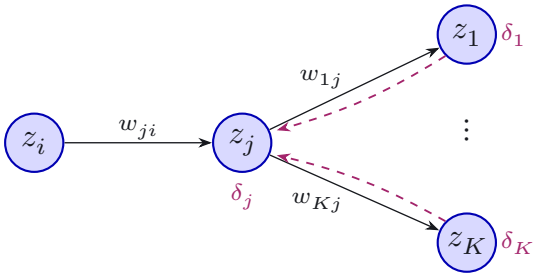
- a_j affects E_n only through the units k it feeds

The error of a hidden unit



- a_j affects E_n only through the units k it feeds
- their errors δ_k come back along the same arrows

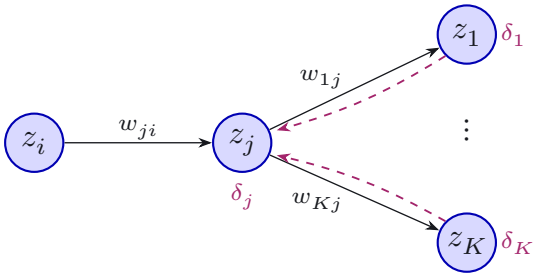
The error of a hidden unit



- a_j affects E_n only through the units k it feeds
- their errors δ_k come back along the same arrows
- chain rule:

$$\delta_j = h'(a_j) \sum_k w_{kj}\, \delta_k$$

The error of a hidden unit



- a_j affects E_n only through the units k it feeds
- their errors δ_k come back along the same arrows
- chain rule:

$$\delta_j = h'(a_j) \sum_k w_{kj}\, \delta_k$$

Key idea: Errors move backward along the arrows, multiplied by the same weights.

Backpropagation

Algorithm 8.1: Backpropagation

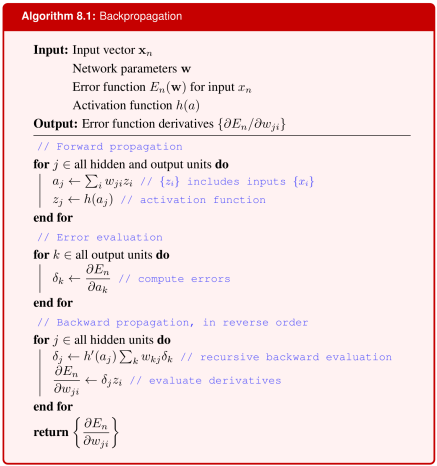
Input: Input vector $\mathbf{x}_n$
Network parameters $\mathbf{w}$
Error function $E_n(\mathbf{w})$ for input x_n
Activation function $h(a)$
Output: Error function derivatives $\{\partial E_n/\partial w_{ji}\}$

```
// Forward propagation
```
for $j \in$ all hidden and output units **do**
  $a_j \leftarrow \sum_i w_{ji} z_i$ `// {z_i} includes inputs {x_i}`
  $z_j \leftarrow h(a_j)$ `// activation function`
end for
```
// Error evaluation
```
for $k \in$ all output units **do**
  $\delta_k \leftarrow \dfrac{\partial E_n}{\partial a_k}$ `// compute errors`
end for
```
// Backward propagation, in reverse order
```
for $j \in$ all hidden units **do**
  $\delta_j \leftarrow h'(a_j) \sum_k w_{kj} \delta_k$ `// recursive backward evaluation`
  $\dfrac{\partial E_n}{\partial w_{ji}} \leftarrow \delta_j z_i$ `// evaluate derivatives`
end for
return $\left\{ \dfrac{\partial E_n}{\partial w_{ji}} \right\}$

Bishop and Bishop, Algorithm 8.1

1. forward pass: store every a_j and z_j

Backpropagation



Algorithm 8.1: Backpropagation

Input: Input vector $\mathbf{x}_n$
Network parameters $\mathbf{w}$
Error function $E_n(\mathbf{w})$ for input x_n
Activation function $h(a)$

Output: Error function derivatives $\{\partial E_n/\partial w_{ji}\}$

```
// Forward propagation
```

for $j \in$ all hidden and output units **do**

$a_j \leftarrow \sum_i w_{ji} z_i$ // $\{z_i\}$ includes inputs $\{x_i\}$

$z_j \leftarrow h(a_j)$ // activation function

end for

```
// Error evaluation
```

for $k \in$ all output units **do**

$\delta_k \leftarrow \dfrac{\partial E_n}{\partial a_k}$ // compute errors

end for

```
// Backward propagation, in reverse order
```

for $j \in$ all hidden units **do**

$\delta_j \leftarrow h'(a_j) \sum_k w_{kj} \delta_k$ // recursive backward evaluation

$\dfrac{\partial E_n}{\partial w_{ji}} \leftarrow \delta_j z_i$ // evaluate derivatives

end for

return $\left\{ \dfrac{\partial E_n}{\partial w_{ji}} \right\}$

Bishop and Bishop, Algorithm 8.1

1. forward pass: store every a_j and z_j
2. output errors: $\delta_k = y_k - t_k$

Backpropagation

Algorithm 8.1: Backpropagation

Input: Input vector $\mathbf{x}_n$
Network parameters $\mathbf{w}$
Error function $E_n(\mathbf{w})$ for input x_n
Activation function $h(a)$

Output: Error function derivatives $\{\partial E_n/\partial w_{ji}\}$

```
// Forward propagation
```
for $j \in$ all hidden and output units **do**
  $a_j \leftarrow \sum_i w_{ji} z_i$ // $\{z_i\}$ includes inputs $\{x_i\}$
  $z_j \leftarrow h(a_j)$ // activation function
end for
```
// Error evaluation
```
for $k \in$ all output units **do**
  $\delta_k \leftarrow \dfrac{\partial E_n}{\partial a_k}$ // compute errors
end for
```
// Backward propagation, in reverse order
```
for $j \in$ all hidden units **do**
  $\delta_j \leftarrow h'(a_j) \sum_k w_{kj} \delta_k$ // recursive backward evaluation
  $\dfrac{\partial E_n}{\partial w_{ji}} \leftarrow \delta_j z_i$ // evaluate derivatives
end for

return $\left\{ \dfrac{\partial E_n}{\partial w_{ji}} \right\}$

Bishop and Bishop, Algorithm 8.1

1. forward pass: store every a_j and z_j
2. output errors: $\delta_k = y_k - t_k$
3. backward pass: every δ_j, last layer first

Backpropagation

Algorithm 8.1: Backpropagation

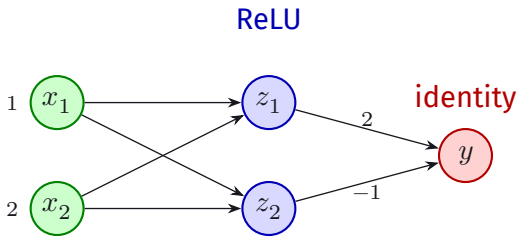
Input: Input vector $\mathbf{x}_n$
Network parameters $\mathbf{w}$
Error function $E_n(\mathbf{w})$ for input x_n
Activation function $h(a)$
Output: Error function derivatives $\{\partial E_n/\partial w_{ji}\}$

```
// Forward propagation
```
for $j \in$ all hidden and output units **do**
  $a_j \leftarrow \sum_i w_{ji} z_i$ `// {z_i} includes inputs {x_i}`
  $z_j \leftarrow h(a_j)$ `// activation function`
end for
```
// Error evaluation
```
for $k \in$ all output units **do**
  $\delta_k \leftarrow \dfrac{\partial E_n}{\partial a_k}$ `// compute errors`
end for
```
// Backward propagation, in reverse order
```
for $j \in$ all hidden units **do**
  $\delta_j \leftarrow h'(a_j) \sum_k w_{kj} \delta_k$ `// recursive backward evaluation`
  $\dfrac{\partial E_n}{\partial w_{ji}} \leftarrow \delta_j z_i$ `// evaluate derivatives`
end for
return $\left\{ \dfrac{\partial E_n}{\partial w_{ji}} \right\}$

Bishop and Bishop, Algorithm 8.1

1. forward pass: store every a_j and z_j
2. output errors: $\delta_k = y_k - t_k$
3. backward pass: every δ_j, last layer first
4. every derivative is one product $\delta_j z_i$

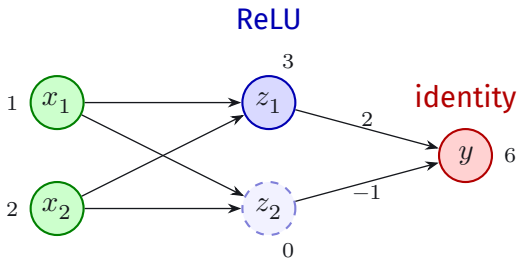
Example: a small network by hand



$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \text{target } t = 4$$

Note: No biases here: no constants 1, and no column 0 in $\mathbf{W}^{(l)}$.

Example: a small network by hand

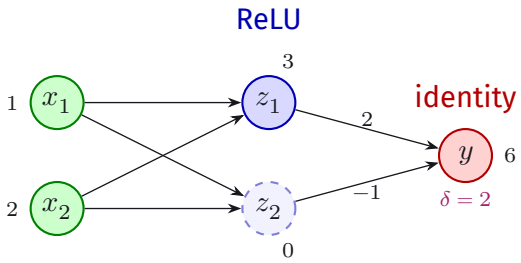


$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \text{target } t = 4$$

- forward:
  $\mathbf{z} = \mathrm{ReLU}(3, -1) = (3, 0), \; y = 6$

Note: No biases here: no constants 1, and no column 0 in $\mathbf{W}^{(l)}$.

Example: a small network by hand

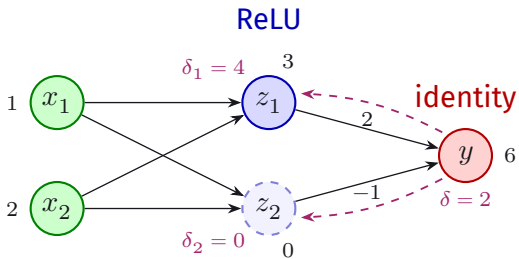


$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \text{target } t = 4$$

- forward: $\mathbf{z} = \mathrm{ReLU}(3, -1) = (3, 0),\ y = 6$
- output: $\delta = y - t = 2$

Note: No biases here: no constants 1, and no column 0 in $\mathbf{W}^{(l)}$.

Example: a small network by hand



$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \text{target } t = 4$$

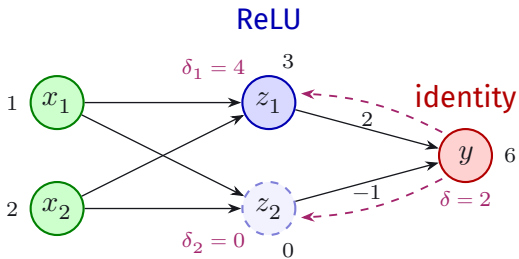
- forward:
  $\mathbf{z} = \mathrm{ReLU}(3, -1) = (3, 0), \; y = 6$
- output: $\delta = y - t = 2$
- hidden (one output, so one term):

$$\delta_j = h'(a_j) \sum_k w_{kj}^{(2)} \, \delta_k = h'(a_j) \, w_{1j}^{(2)} \, \delta$$

$\delta_1 = 4, \; \delta_2 = 0$ (why?)

Note: No biases here: no constants 1, and no column 0 in $\mathbf{W}^{(l)}$.

Example: a small network by hand



$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \text{target } t = 4$$

$$\frac{\partial E_n}{\partial \mathbf{W}^{(1)}} = \begin{bmatrix} 4 & 8 \\ 0 & 0 \end{bmatrix}$$

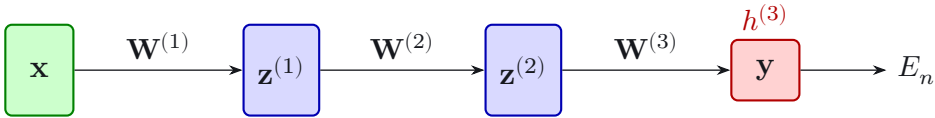
- forward:
  $\mathbf{z} = \mathrm{ReLU}(3, -1) = (3, 0), \; y = 6$
- output: $\delta = y - t = 2$
- hidden (one output, so one term):

$$\delta_j = h'(a_j) \sum_k w_{kj}^{(2)} \, \delta_k = h'(a_j) \, w_{1j}^{(2)} \, \delta$$

  $\delta_1 = 4, \; \delta_2 = 0$ (why?)
- derivatives:
  $\partial E_n / \partial \mathbf{W}^{(1)} = (\delta_1, \delta_2)^\mathsf{T} (x_1, x_2)$

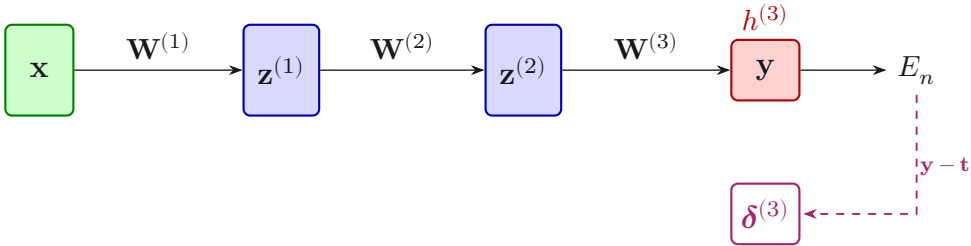
Note: No biases here: no constants 1, and no column 0 in $\mathbf{W}^{(l)}$.

The backward pass, block by block



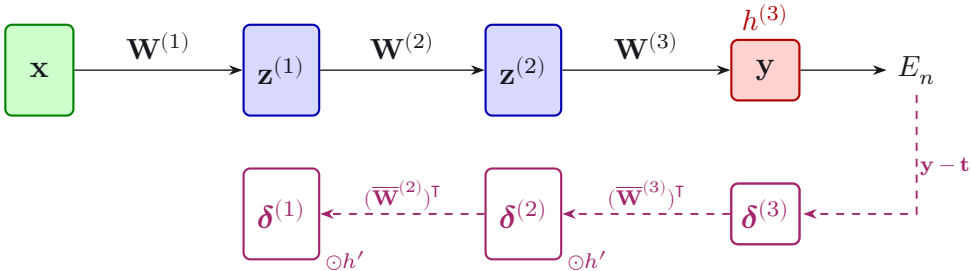
Note: New: a block with a magenta outline holds the errors $\boldsymbol{\delta}^{(l)}$ of the layer above it.

The backward pass, block by block



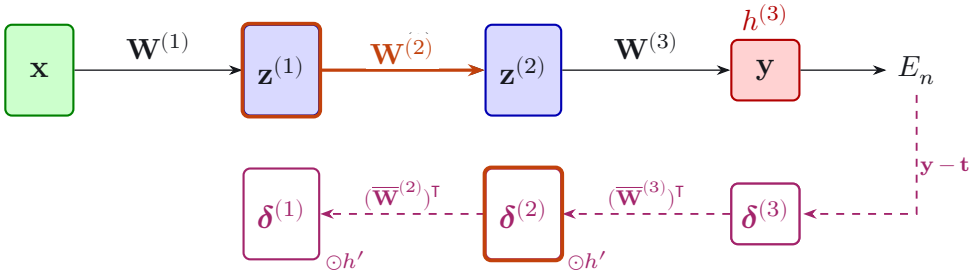
Note: New: a block with a magenta outline holds the errors $\boldsymbol{\delta}^{(l)}$ of the layer above it.

The backward pass, block by block



- each step back: multiply by $(\overline{\mathbf{W}}^{(l)})^\mathsf{T}$, then by h' entrywise
- $\overline{\mathbf{W}}^{(l)}$: drop the bias column, as there is no δ_0 for $z_0 = 1$

The backward pass, block by block



- each step back: multiply by $(\overline{\mathbf{W}}^{(l)})^\mathsf{T}$, then by h' entrywise
- $\overline{\mathbf{W}}^{(l)}$: drop the bias column, as there is no δ_0 for $z_0 = 1$
- derivatives for $\mathbf{W}^{(l)}$ use both ends of its arrow: $\boldsymbol{\delta}^{(l)}\left(\mathbf{z}^{(l-1)}\right)^\mathsf{T}$

What backpropagation costs

- forward pass: one multiply-add per weight, about P
- backward pass: the products $\boldsymbol{\delta}^{(l)}(\mathbf{z}^{(l-1)})^{\mathsf{T}}$ cost about P, the steps back less than P

method	cost, in evaluations of the network	for 784–128–10
finite differences	$P+1$	$101{,}771$
backpropagation	about 3	about 3

What backpropagation costs

- forward pass: one multiply-add per weight, about P
- backward pass: the products $\boldsymbol{\delta}^{(l)}(\mathbf{z}^{(l-1)})^{\mathsf{T}}$ cost about P, the steps back less than P

method	cost, in evaluations of the network	for 784–128–10
finite differences	$P+1$	$101{,}771$
backpropagation	about 3	about 3

Watch out: Training must store every $\mathbf{a}^{(l)}$ and $\mathbf{z}^{(l)}$ until the backward pass. Prediction does not need to.

Plan for today

• Recap	2 min
• Large networks and their parameters	6 min
• Error functions	4 min
• Optimization	12 min
• Backpropagation	16 min
• PyTorch	8 min

PyTorch

A Python library for building and training networks:

PyTorch	what it is
`torch.tensor`	an array, like a NumPy array
`E.backward()`	backpropagation: fills `.grad` of each weight
`nn.Linear(q, m)`	one $\mathbf{W}^{(l)}$: a `weight` ($m \times q$) and a `bias` (m)
`nn.ReLU()`	the activation
`nn.Sequential(...)`	layers in order, as in a block diagram
`nn.CrossEntropyLoss()`	error function, softmax included
`torch.optim.Adam`	update rule
`DataLoader`	shuffled mini-batches

- `import torch` and `from torch import nn`

Example: backprop for the example from earlier

```
import torch                # the example, no biases
x = torch.tensor([1., 2.])
t = torch.tensor(4.)
W1 = torch.tensor([[1., 1.], [1., -1.]],
                  requires_grad=True)
w2 = torch.tensor([2., -1.], requires_grad=True)
z = torch.relu(W1 @ x)      # forward pass
y = w2 @ z
E = 0.5 * (y - t) ** 2
E.backward()                # backward pass
print(W1.grad)   # tensor([[4., 8.], [0., 0.]])
print(w2.grad)   # tensor([6., 0.])
```

Python: model, error, optimizer, data

```
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

model = nn.Sequential(           # 784 -> 128 -> 10
    nn.Linear(784, 128), nn.ReLU(),
    nn.Linear(128, 10))          # no softmax layer
lossf = nn.CrossEntropyLoss()    # softmax + log loss
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

data = TensorDataset(X_train, t_train)  # tensors
loader = DataLoader(data, batch_size=64,
                    shuffle=True)
```

- `X_train`: an $N \times 784$ tensor; `t_train`: N class labels $0, \ldots, 9$

Python: the training loop

```
for epoch in range(20):
    for xb, tb in loader:           # one mini-batch
        E = lossf(model(xb), tb)    # forward pass, error
        opt.zero_grad()             # clear old gradients
        E.backward()                # backward pass
        opt.step()                  # update the weights
```

- one line for each box of the training loop
- `E.backward()` computes all $101{,}770$ derivatives by backpropagation

Python: the training loop

```
for epoch in range(20):
    for xb, tb in loader:           # one mini-batch
        E = lossf(model(xb), tb)    # forward pass, error
        opt.zero_grad()             # clear old gradients
        E.backward()                # backward pass
        opt.step()                  # update the weights
```

- one line for each box of the training loop
- `E.backward()` computes all $101{,}770$ derivatives by backpropagation

Watch out: `backward()` adds to `.grad`, so `zero_grad()` must come first.

Summary

- A layer with q inputs and m units has $m(q+1)$ parameters
- Training minimizes $E(\mathbf{w})$ by mini-batch SGD or Adam, given the gradient
- Backpropagation gives the gradient: one pass forward, one pass back; each derivative is $\delta_j z_i$
- In PyTorch, `E.backward()` does the backpropagation and `opt.step()` the update